



结构化并发编程技巧

北京理工大学计算机学院
金旭亮

概述



所谓“结构化并发 (Structured Concurrency)”，其实就是指多个虚拟线程之间的相互协作。



比如，“A线程结束之后，将结果传给B线程处理”，或者是“A、B、C三个线程同时运行，只要有一个线程得到结果就行了，其他还没有得到结果的线程就取消”，诸如此类的，都是典型的“结构化并发”开发场景。



JDK中为实现上述目的提供了相应的组件，与RxJava或Kotlin的协程相比，虽然不如后者功能完备，但也基本上够用了。

一个示例，揭示多线程程序存在的问题.....



```
ExecutorService es = ...;  
Future<Text> f1 = es.submit(PageReader::readText);  
Future<Link> f2 = es.submit(PageReader::readLinks);  
Page page = new Page(f1.get(1, TimeUnit.SECONDS),  
                      f2.get(1, TimeUnit.SECONDS));
```

上述代码中，对Future.get()方法的调用是阻塞的。

如果所有调用都能顺利地完成，则上述代码不会有任何问题，但如果f1.get()这句迟迟不能返回，或者抛出了异常，都会导致f2.get()没有机会执行，或者可能会居于一种“未知”的状态中，这些都为程序调试带来了困难。

实际开发，呼吁对线程协作及异常处理提供一致的技术处理方案.....

基于虚拟线程的“结构化并发”



“结构化并发”试图规范多个虚拟线程的行动，将相关的线程“组合”在一起集体行动，这个组合，被称为“StructuredTaskScope”。



虚拟线程归属于“StructuredTaskScope”。一个“StructuredTaskScope”可以有多个线程，当所有线程运行结束，这个Scope销毁，Scope负责协调它所包容的多个线程之间的协作关系，比如，它可以等待运行速度不一致的A、B、C三个线程都运行结束，然后启动D线程进行收尾，D线程运行结束后，整个Scope销毁。



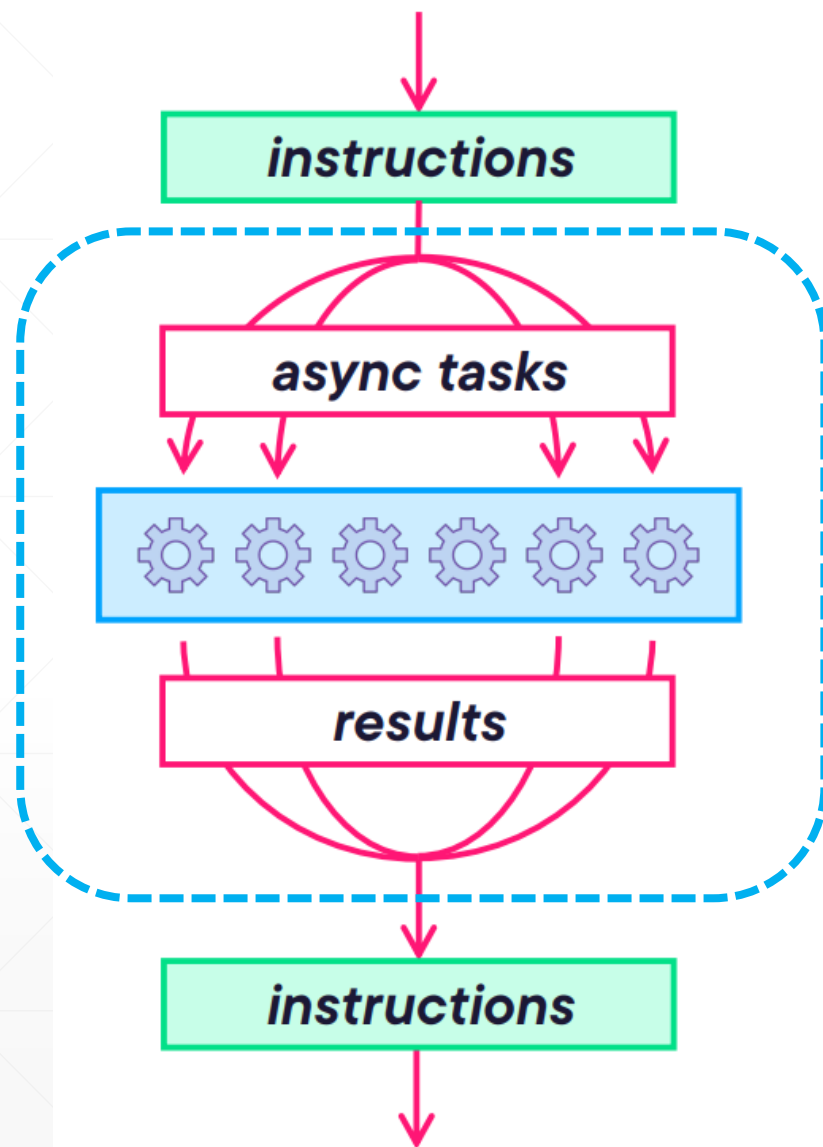
我们还可以设定StructuredTaskScope，当虚拟线程在运行过程中抛出未捕获的异常时，自动运行某个处理程序.....

结构化并发的程序执行流程

如右图所示，虚线框中的部分，就是“结构化并发”。

程序创建多个虚拟线程并发执行多个异步任务，然后再汇总这些线程的结果，进行后继处理.....

不管是线程正常运行结束，或者是某个线程由于出现未捕获异常而提前中止，只要程序执行流程离开“结构化并发”的边界，这里面的线程都会被“清理”干净。



StructuredTaskScope当前是“预览”特性



```
import java.util.concurrent.StructuredTaskScope;

public class UseStructuredTaskScope {
    public static void main(String[] args) {
        try (var scope = new StructuredTaskScope()) {
            // ...
        }
    }
}
```

java.util.concurrent.StructuredTaskScope is a preview API and may be removed in a future release

[Set language level to 21 \(Preview\) - String templates, unnamed classes and instance main methods etc.](#) Alt+Shift+Enter [More actions...](#) Alt+Enter

© java.util.concurrent.StructuredTaskScope<T>

public StructuredTaskScope()

Creates an unnamed structured task scope that creates virtual threads. The task scope is owned by the current thread.

Implementation This constructor is equivalent to invoking the 2-arg constructor with a name of `null`

Requirements: and a thread factory that creates virtual threads.

< 21 >

IntelliJ IDEA在编译上述代码时，会给出相应提示，直接点击相应链接，相关配置就完成了。

准备工作-休眠指定的时长



在MyAsyncTask类中，放置了一些用于测试的代码。

```
//休眠指定的时间
private static void sleepFor(int amount, ChronoUnit unit) {
    try {
        Thread.sleep(Duration.of(amount, unit));
    } catch (InterruptedException e) {
        throw new RuntimeException(e);
    }
}
```

这个方法主要用于“拖慢”线程的执行，以便能模拟真实项目中耗时不同的异步任务

准备工作-异步任务生成器



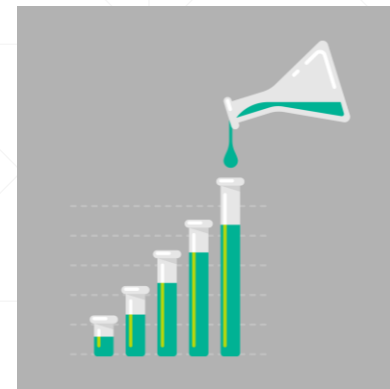
```
private static Random random = new Random();

//用于模拟"成功的"异步任务
public static String processSuccessAsyncTask(String taskInfo, int origin, int bound) {
    int randomTime = random.nextInt(origin, bound);
    sleepFor(randomTime, ChronoUnit.MILLIS);
    return "[" + taskInfo + "]"运行结束, " + Thread.currentThread() + " 耗时: " + randomTime + "ms";
}

//用于模拟"失败的"异步任务
public static String processFailureAsyncTask(String taskInfo, int origin, int bound) {
    int randomTime = random.nextInt(origin, bound);
    sleepFor(randomTime, ChronoUnit.MILLIS);
    throw new RuntimeException "[" + taskInfo + "]"运行过程中抛出了异常");
}
```

上述两个静态方法，用于生成测试用的异步任务，它调用了前面介绍的sleepFor方法休眠指定范围内的时间，以模拟异步任务的执行过程。

准备工作-用于测试的异步任务



```
public static Callable<String> task1 = () ->
    MyAsyncTask.processSuccessAsyncTask("异步任务一", 60, 150);
public static Callable<String> task2 = () ->
    MyAsyncTask.processSuccessAsyncTask("异步任务二", 80, 100);
public static Callable<String> task3 = () ->
    MyAsyncTask.processSuccessAsyncTask("异步任务三", 300, 1100);

public static Callable<String> failureTask = () ->
    MyAsyncTask.processFailureAsyncTask("出错的异步任务", 10, 100);
```

示例中创建了三个可以成功完成的异步任务，一个会失败的异步任务，这些异步任务都会随机休眠相应的时间。后两个方法参数定义了休眠时间的上限和下限。

核心组件—StructuredTaskScope



StructuredTaskScope是异步任务的容器，注意到它实现了AutoCloseable接口，这意味着它可以放到try-with-resource结构中，其close()方法会被自动地调用。

```
public class StructuredTaskScope<T> implements AutoCloseable {  
    ...  
    public <U extends T> Subtask<U> fork(Callable<? extends U> task) { }  
    public StructuredTaskScope<T> join() throws InterruptedException  
    public void close(){}  
    ...  
}
```



StructuredTaskScope最重要的方法有两个：

① fork(): 用于启动一个异步任务，这个任务本身由Callable对象表达，是由虚拟线程负责执行的。这个方法可以调用多次，启动多个异步任务。

② join(): 用于等待所有异步任务的完成。

核心组件-SubTask

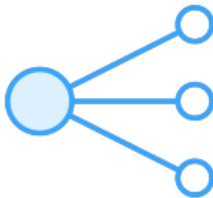


```
Subtask
├── State
│   ├── UNAVAILABLE: State
│   ├── SUCCESS: State
│   └── FAILED: State
├── task(): Callable<? extends T>
├── state(): State
├── get(): T
└── exception(): Throwable
```

`fork()` 方法返回一个 `SubTask<T>` 实例，`SubTask` 其实是一个派生自 JDK 中 `Supplier<T>` 的泛型接口，这个接口定义了一些非常有用的成员：

- `task()` 方法返回一个 `Callable` 对象，就是前面传给 `fork()` 方法的那个。
- `get()` 方法返回当前异步任务的结果。
- `exception()` 方法返回异步任务执行过程抛出的那些未经捕获的异常。
- `state()` 方法返回异步任务的状态，这是一个包容有三个成员的枚举值，具体含义见下页 PPT。

异步任务的状态



SubTask.state()方法返回异步任务的状态，包容三个值：

状态值	说 明
SUCCESS	任务成功完成，现在可以调用Subtask.get()方法得到任务结果。
FAILED	任务失败，可以调用Subtask.exception()方法获取任务执行过程中抛出的异常对象。
UNAVAILABLE	表示此任务未能执行结束，因此，get()和exception()方法的调用是无效的。

典型场景-多任务成功并行执行



//所有任务均成功

```
private static void allSuccess() {
    try (var scope = new StructuredTaskScope<String>()) {
        //启动三个异步任务
        var t1 = scope.fork(MyAsyncTask.task1);
        var t2 = scope.fork(MyAsyncTask.task2);
        var t3 = scope.fork(MyAsyncTask.task3);
        //等待所有异步任务的完成
        scope.join();
        //检查每个异步任务的状态, 如果成功, 取出结果
        System.out.println("任务一状态: " + t1.state());
        if (t1.state() == StructuredTaskScope.Subtask.State.SUCCESS) {
            System.out.println(t1.get());
        }
        System.out.println("任务二状态: " + t2.state());
        if (t2.state() == StructuredTaskScope.Subtask.State.SUCCESS) {
            System.out.println(t2.get());
        }
        System.out.println("任务三状态: " + t3.state());
        if (t3.state() == StructuredTaskScope.Subtask.State.SUCCESS) {
            System.out.println(t3.get());
        }
    } catch (InterruptedException e) {
        System.out.println(e.getMessage());
    }
}
```

左图所示为典型的使用虚拟线程的异步编程模式, 将scope放到try()表达式中, 调用fork()方法启动异步任务的执行, 然后调用join()方法等待其完成。

任务完成后, 检查其状态, 输出其结果。

从程序输出中可以看到, 异步任务是在ForkJoinPool中的线程执行的, 不是主线程。

```
Run UseStructuredTaskScope x
"C:\Program Files\Java\jdk-21\bin\java.exe" --enable-preview "-javaagent:C:\Program Files\Java\jdk-21\bin\javaagent.jar"
Thread[#1,main,5,main]:main()开始
任务一状态: SUCCESS
[异步任务一]运行结束, VirtualThread[#30]/runnable@ForkJoinPool-1-worker-1 耗时: 115ms
任务二状态: SUCCESS
[异步任务二]运行结束, VirtualThread[#32]/runnable@ForkJoinPool-1-worker-1 耗时: 96ms
任务三状态: SUCCESS
[异步任务三]运行结束, VirtualThread[#33]/runnable@ForkJoinPool-1-worker-1 耗时: 992ms
Thread[#1,main,5,main]:main()结束
Process finished with exit code 0
```

典型场景-分步执行的异步任务



//分阶段执行的任务

```
private static void stepByStep() {
    try (var scope = new StructuredTaskScope<String>()) {
        var t1 = scope.fork(MyAsyncTask.task1);
        var t2 = scope.fork(MyAsyncTask.task2);
        scope.join();
        System.out.println("\n第一阶段完成...");
        System.out.println("任务一状态: " + t1.state());
        if (t1.state() == StructuredTaskScope.Subtask.State.SUCCESS) {
            System.out.println(t1.get());
        }
        System.out.println("任务二状态: " + t2.state());
        if (t2.state() == StructuredTaskScope.Subtask.State.SUCCESS) {
            System.out.println(t2.get());
        }
        var t3 = scope.fork(MyAsyncTask.task3);
        scope.join();
        System.out.println("\n第二阶段完成...");
        System.out.println("任务三状态: " + t3.state());
        if (t3.state() == StructuredTaskScope.Subtask.State.SUCCESS) {
            System.out.println(t3.get());
        }
    } catch (InterruptedException e) {
        System.out.println(e.getMessage());
    }
}
```

多次调用fork()和join(), 可以创建需要分步执行的异步任务。

```
"C:\Program Files\Java\jdk-21\bin\java.exe" --enable-preview "-javaagent:C:\Progra
Thread[#1,main,5,main]:main()开始
```

第一阶段完成...

任务一状态: SUCCESS

[异步任务一]运行结束, VirtualThread[#30]/runnable@ForkJoinPool-1-worker-2 耗时: 117ms

任务二状态: SUCCESS

[异步任务二]运行结束, VirtualThread[#32]/runnable@ForkJoinPool-1-worker-2 耗时: 99ms

第二阶段完成...

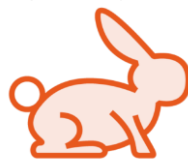
任务三状态: SUCCESS

[异步任务三]运行结束, VirtualThread[#35]/runnable@ForkJoinPool-1-worker-2 耗时: 808ms

Thread[#1,main,5,main]:main()结束

Process finished with exit code 0

典型场景—最先抵达者胜



//只要有一个任务成功，就自动退出且关闭，其他任务都没有必要运行了

```
private static void shutdownOnSuccesss() {  
  
    try (var scope = new StructuredTaskScope.ShutdownOnSuccess()) {  
        var t1 = scope.fork(MyAsyncTask.task1);  
        var t2 = scope.fork(MyAsyncTask.task2);  
        var t3 = scope.fork(MyAsyncTask.task3);  
        scope.join();  
        //余下代码略...  
    } catch (InterruptedException e) {  
        System.out.println(e.getMessage());  
    } catch (Exception e) {  
        System.out.println(e.getMessage());  
    }  
}
```

任务二先完成，所以，任务一和任务三都不需要再执行，其状态为UNAVAILABLE。



```
"C:\Program Files\Java\jdk-21\bin\java.exe" --enable-preview "-javaagent:C:\Program Files\Java\jdk-21\bin\javaagent.jar" -Djdk.attach.allowAttachSelf --add-exports java.base/sun.nio.ch=ALL-UNNAMED --add-exports java.base/sun.util.resources=ALL-UNNAMED --add-exports java.desktop/com.apple.laf=ALL-UNNAMED --add-exports java.desktop/sun.awt=ALL-UNNAMED --add-exports java.desktop/sun.awt.image=ALL-UNNAMED --add-exports java.instrument/sun.instrument=ALL-UNNAMED --add-exports java.management/sun.management=ALL-UNNAMED --add-exports java.naming/com.sun.naming=ALL-UNNAMED --add-exports java.rmi/sun.rmi.registry=ALL-UNNAMED --add-exports java.security.sasl/javax.sasl.provider.impl=ALL-UNNAMED --add-exports java.transaction/sun.transaction=ALL-UNNAMED --add-exports java.transaction.spi=ALL-UNNAMED --add-exports java.xml.crypto/sun.xml.crypto.provider=ALL-UNNAMED --add-exports java.xml.crypto/sun.xml.crypto.provider=ALL-UNNAMED --add-exports java.xml.crypto/sun.xml.crypto.provider=ALL-UNNAMED --add-exports java.xml.crypto/sun.xml.crypto.provider=ALL-UNNAMED --add-exports java.xml.crypto/sun.xml.crypto.provider=ALL-UNNAMED --add-exports java.xml.crypto/sun.xml.crypto.provider=ALL-UNNAMED --add-exports java.xml.crypto/sun.xml.crypto.provider=ALL-UNNAMED --add-exports java.xml.crypto/sun.xml.crypto.provider=ALL-UNNAMED --add-exports java.xml.crypto/sun.xml.crypto.provider=ALL-UNNAMED --add-exports java.xml.crypto/sun.xml.crypto.provider=ALL-UNNAMED --add-exports java.xml.crypto/sun.xml.crypto.provider=ALL-UNNAMED --add-exports java.xml.crypto/sun.xml.crypto.provider=ALL-UNNED  
Thread[#1,main,5,main]:main()开始  
任务一状态: UNAVAILABLE  
任务二状态: SUCCESS  
[异步任务二]运行结束, VirtualThread[#32]/runnable@ForkJoinPool-1-worker-2 耗时: 93ms  
任务三状态: UNAVAILABLE  
Thread[#1,main,5,main]:main()结束  
  
Process finished with exit code 0
```


典型场景——一招不慎，满盘皆输



//只要有一个任务失败，就自动退出且关闭，其他任务都没有必要进行了

```
private static void shutdownOnFailure() {  
    try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {  
  
        var t1 = scope.fork(MyAsyncTask.task1);  
        var t2 = scope.fork(MyAsyncTask.task2);  
        var t3 = scope.fork(MyAsyncTask.failureTask);  
  
        scope.join();  
  
        //以下代码略...  
        System.out.println("任务三状态: " + t3.state());  
        if (t3.state() == StructuredTaskScope.Subtask.State.SUCCESS) {  
            System.out.println(t3.get());  
        } else {  
            System.out.println(t3.exception());  
        }  
        //以下代码略...  
    } catch (InterruptedException e) {  
        System.out.println(e.getMessage());  
    } catch (Exception e) {  
        System.out.println(e.getMessage());  
    }  
}
```

当有一个任务失败时，调用其
`exception()`方法捕获其异常

```
"C:\Program Files\Java\jdk-21\bin\java.exe" --enable-preview  
Thread[#1,main,5,main]:main()开始  
任务一状态: UNAVAILABLE  
任务二状态: UNAVAILABLE  
任务三状态: FAILED  
java.lang.RuntimeException: [出错的异步任务]运行过程中抛出了异常  
Thread[#1,main,5,main]:main()结束
```



在实际开发中，可以从StructuredTaskScope派生出子类，重写其handleComplete()方法，并可以添加一些额外的成员，以封装业务逻辑，方便使用。

本讲示例中，为CustomScope添加了三个属性，分别代表任务执行结果，抛出的异常，以及任务执行过程中是否出错.....

自定义Scope外部的访问接口

```
//自定义StructuredTaskScope
public class CustomScope extends StructuredTaskScope<String> { 2 usages
    //存储异步任务的结果
    private volatile Collection<String> taskResults = new ConcurrentLinkedDeque<>(); 2 usages
    public List<String> getTaskResults() { 2 usages
        return taskResults.stream().toList();
    }
    //存储异步任务可能发生的异常
    private volatile Collection<Throwable> exceptions = new ConcurrentLinkedQueue<>(); 3 usages
    public List<Throwable> getExceptions() { 2 usages
        return exceptions.stream().toList();
    }
    //用于供外界查询是否有失败的任务
    private volatile boolean hasFailure = false; 4 usages
    public boolean isHasFailure() { return hasFailure; }
    public void setHasFailure(boolean hasFailure) { this.hasFailure = hasFailure; }

    @Override 5 usages
    protected void handleComplete(Subtask<? extends String> subtask) {...}
}
```

注意选择线程安全的数据结构

这里封装业务逻辑

自定义Scope封装的业务逻辑



```
@Override
protected void handleComplete(Subtask<? extends String> subtask) {

    switch (subtask.state()) {
        case UNAVAILABLE -> {
            hasFailure = true;
            this.exceptions.add(new IllegalStateException("异步任务无法执行"));
        }
        case SUCCESS -> this.taskResults.add(subtask.get());
        case FAILED -> {
            hasFailure = true;
            this.exceptions.add(subtask.exception());
        }
    }
}
```

测试自定义Scope-全部成功



```
private static void allSuccess() {
    try (var scope = new CustomScope()) {
        var t1 = scope.fork(MyAsyncTask.task1);
        var t2 = scope.fork(MyAsyncTask.task2);
        var t3 = scope.fork(MyAsyncTask.task3);
        scope.join();
        if (scope.isHasFailure()) {
            System.out.println("出错了");
            scope.getExceptions().forEach(System.err::println);
        } else {
            System.out.println("程序运行的结果为: ");
            scope.getTaskResults().forEach(System.err::println);
        }
    } catch (InterruptedException e) {
        throw new RuntimeException(e);
    }
}
```

```
"C:\Program Files\Java\jdk-21\bin\java.exe" --enable-preview "-javaagent:C:\Program
程序运行的结果为:
[异步任务一]运行结束, VirtualThread[#30]/runnable@ForkJoinPool-1-worker-1 耗时: 61ms
[异步任务二]运行结束, VirtualThread[#32]/runnable@ForkJoinPool-1-worker-1 耗时: 90ms
[异步任务三]运行结束, VirtualThread[#33]/runnable@ForkJoinPool-1-worker-1 耗时: 912ms

Process finished with exit code 0
```

测试自定义Scope-部分失败



```
private static void someFailure() {  
    try (var scope = new CustomScope()) {  
        scope.fork(MyAsyncTask.task1);  
        scope.fork(MyAsyncTask.failureTask);  
        scope.fork(MyAsyncTask.task3);  
        scope.join();  
        if (scope.isHasFailure()) {  
            System.out.println("出错了");  
            scope.getExceptions().forEach(System.err::println);  
        } else {  
            System.out.println("程序运行的结果为: ");  
            scope.getTaskResults().forEach(System.err::println);  
        }  
    } catch (InterruptedException e) {  
        throw new RuntimeException(e);  
    }  
}
```

```
Run UseCustomScope x  
"C:\Program Files\Java\jdk-21\bin\java.exe" --enable-preview  
出错了  
java.lang.RuntimeException: [出错的异步任务]运行过程中抛出了异常  
Process finished with exit code 0
```